

**SCHOOL OF COMPUTING
PHENIKAA UNIVERSITY**

-----o0o-----



PROJECT REPORT

COURSE: SOFTWARE ARCHITECTURE

PROJECT TITLE: CINEMA MANAGEMENT

Group 14

Student Name	ID
Nguyen Que Bac	23010574
Hoang Tuan Kiet	23010517
Do Bao Long	23010561

Course Instructor: Vu Quang Dung

Ha Noi, February 5, 2026

Contents

1. Cover Page & Info	4
2. Executive Summary	4
3. Project Requirements & Goals	4
3.1 Core Functional Requirement	4
3.2 Key Quality Attributes (Architectural Goals – Non-functional Requirements)	5
3.3 Proposed Model of ASR	6
3.4 Use Case Modeling	6
4. Architectural Design & Implementation	9
4.1 Architectural Pattern: Layered Monolithic Architecture (MVC)	9
4.2 Technical Stack and Data Model	17
4.3 API Endpoints	18
4.4 Overall Architecture – Layered Monolith	20
4.5 Detailed Architecture	21
4.6 Implementation: Server-Side Logic (Laravel)	24
4.7 Implementation: Client-Side Logic (Laravel Blade / Server-Side Rendering)	27
5. Testing & Verification	29
6. Conclusion & Reflection	30
6.1 Lessons Learned	30
6.2 Future Improvements	30
References	32

List of Tables

Table 1: Core Funtional Requirement	5
Table 2: User API	18
Table 3: Movie API	18
Table 4: Showtime API	18
Table 5: Booking API.....	19
Table 6: Seat API.....	19
Table 7: Admin API.....	19
Table 8: Testing	29

List of Figures

Figure 1: Use Case Diagram.....	8
Figure 2: System Context Diagram	10
Figure 3: Container Diagram	12
Figure 4: Backend Component Diagram	15

1. Cover Page & Info

- **Project Title:** Cinema Management
- **Course Name:** Software Architecture
- **Student Info:** [Nguyen Que Bac], [23010574]; [Hoàn Tuan Kiet], [23010517]; ; [Do Bao Long], [23010561]
- **Date:** February 5, 2026

2. Executive Summary

This project focuses on the design and implementation of a web-based Cinema Management System that supports movie scheduling, ticket booking, seat management, and administrative operations. The system is built using a layered architecture with a centralized database to ensure efficient data handling and consistency.

The final result is a functional application that enables users to book tickets conveniently while allowing administrators to manage cinema operations and view statistical reports, successfully meeting key quality attributes such as usability, reliability, and maintainability.

3. Project Requirements & Goals

This section defines the scope and the key architectural drivers for the project.

3.1 Core Functional Requirement

ID	Description
FR-01	The system must allow users to register and authenticate using a unique account.
FR-02	The system must allow users to view the list of available movies and their details (title, genre, duration, and description).
FR-03	The system must allow users to view movie showtimes based on date and cinema room.
FR-04	The system must allow users to select seats and create a ticket booking for a specific showtime.

FR-05	The system must validate seat availability and prevent double booking.
FR-06	The system must allow users to confirm, cancel, or view their booking history.
FR-07	The system must allow administrators to manage movies, showtimes, cinema rooms, and seating layouts.
FR-08	The system must allow administrators to view statistical reports including ticket sales, revenue, and booking status.

Table 1: Core Funtional Requirement

3.2 Key Quality Attributes (Architectural Goals – Non-functional Requirements)

- **Performance:** The system must provide responsive user interactions, particularly for seat selection and booking operations. Seat availability and booking confirmation should be processed with minimal latency to ensure a smooth user experience.
- **Availability:** The system should remain available during peak usage periods, such as evenings and weekends, ensuring that users can access showtimes, book tickets, and perform administrative operations without service interruption.
- **Data Consistency:** The system must guarantee consistent and accurate booking data, preventing double seat reservations and ensuring that all transactions reflect the correct system state.
- **Modifiability:** The system architecture follows a layered monolithic design with clear separation between presentation, business logic, and data access layers. This structure allows new features—such as online payments, promotions, or additional reporting functions—to be added with minimal impact on existing components.
- **Security:** The system must protect user data and booking information through proper authentication, authorization, and secure data handling mechanisms.

3.3 Proposed Model of ASR

ASR: Performance and Scalability (Non-functional Requirement)

- **Statement:** The system must support a large number of concurrent users (e.g., 500–1000 users) during peak booking periods while maintaining acceptable response times (under 2 seconds) for seat selection and booking confirmation.
- **Impact:** This requirement influences the architectural choice. It drives the use of an optimized monolithic architecture with efficient database schema design, caching mechanisms (e.g., Redis for frequently accessed data like showtimes and seat availability), and proper concurrency control to ensure stable performance under increased load.
- **Rationale:** A monolithic layered MVC architecture is chosen because the system is medium-scale, where simplicity and maintainability are prioritized. At the same time, performance and scalability can be achieved through database optimization, caching, and horizontal scaling if needed. This balances development speed with system reliability during peak usage.

3.4 Use Case Modeling

1. System Boundary

The system boundary

2. Actors

Actors outside the system:

- Customer
- Administrator
- Payment Gateway

3. Top-Level Use Cases

Customer:

- Search Movies
- View Showtimes
- Book Ticket

- Manage Account

Administrator:

- Manage Movies
- Manage Showtimes
- Manage Bookings

- **Connect Actors to Use Cases**

Customer interacts with: Search Movies, View Showtimes, Book Ticket, Manage Account

Administrator interacts with: Manage Movies, Manage Showtimes, Manage Bookings

Payment Gateway interacts with: Secure Payment

4. Detailed Critical Use Case: Book Ticket

Main use case: Book Ticket

Include:

- Select Seats
- Secure Payment

Extend:

- Add Food Combo (optional)

UML Use Case Diagram Description

Actors:

- Customer on the left
- Administrator on the right
- Payment Gateway below

System Boundary: “MovieTicket Online Booking System”

Use Cases:

- Search Movies
- View Showtimes
- Book Ticket

- Include: Select Seats
- Include: Secure Payment
- Extend: Add Food Combo
- Manage Account
- Manage Movies
- Manage Showtimes
- Manage Bookings

Use Case Diagram



Figure 1: Use Case Diagram

4. Architectural Design & Implementation

4.1 Architectural Pattern: Layered Monolithic Architecture (MVC)

The core of the application is based on a layered monolithic architecture following the Model–View–Controller (MVC) pattern. This architectural pattern is suitable for the cinema management system as it clearly separates presentation, business logic, and data access concerns within a single deployable application.

User requests are handled by controllers, which coordinate business logic through service and model layers before interacting with the centralized relational database. This structure simplifies development, improves maintainability, and ensures consistent data processing for critical operations such as seat selection, booking validation, and administrative management.

System Context Diagram (C4 Model – Level 1)

The C4 Model (Level 1: System Context) illustrates the boundary of the Cinema Management System.

Actors:

- Users / Customers: Browse movies, select seats, and book tickets via a web browser.
- Administrators: Manage movies, showtimes, cinema rooms, seats, and view reports.

Interactions:

- All interactions occur via standard HTTP requests and responses.
- The system communicates with a centralized relational database to store and retrieve data related to users, movies, showtimes, bookings, and reports.

External Systems:

None required for core functionality. The design remains simple and cohesive, focusing on reliable cinema operation management.

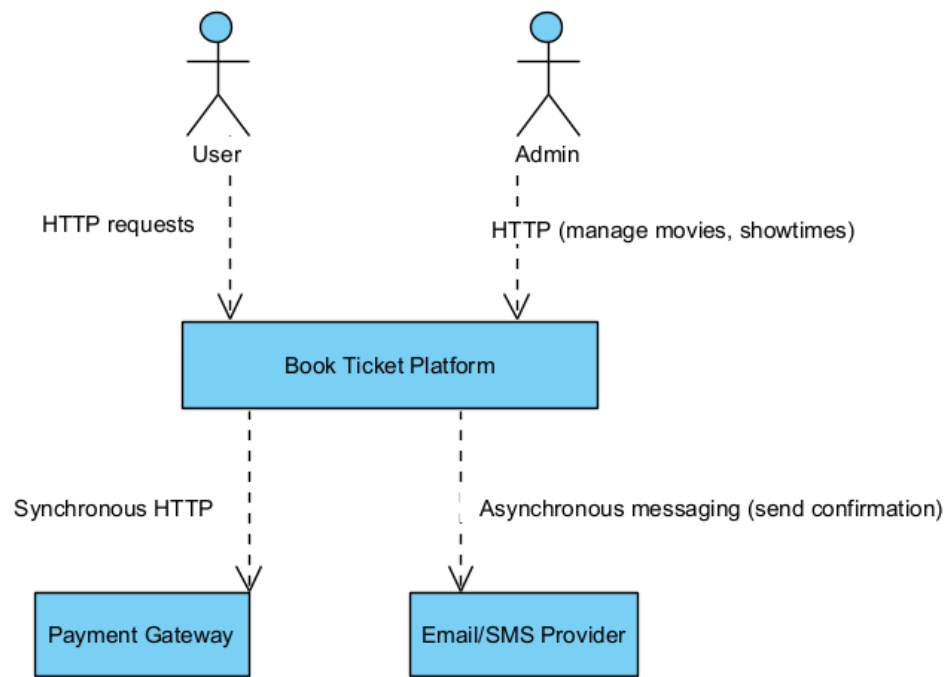


Figure 2: System Context Diagram

Container Diagram (C4 Model – Level 2)

The container diagram provides a closer look at the main components of the system:

Web Browser (User Interface):

- Displays HTML pages rendered via Blade templates.
- Submits forms for booking, account management, and administrative tasks.

Laravel Application (Monolithic Backend):

- Controllers: Handle HTTP requests and coordinate application logic.
- Models / Eloquent ORM: Interact with the database and enforce data relationships.
- Views / Blade Templates: Render dynamic HTML pages for users and administrators.

- **Business Logic:** Seat availability checks, booking processing, report generation.

MySQL Database:

- Stores persistent data for users, movies, showtimes, cinema rooms, seats, bookings, and tickets.
- Ensures data consistency and supports transaction management for concurrent booking requests.

Payment Simulation (Internal Component / Mock Service):

- **Role:** Provides a simulated payment process for testing and demonstration purposes, without connecting to an actual external gateway.
- **Responsibilities:**
 - Mimics the behavior of a real payment gateway (request, validation, response).
 - Returns predefined outcomes (success/failure) to allow the booking workflow to be tested end-to-end.
 - Ensures that the system logic for handling payments, confirmations, and error cases can be verified.
- **Interactions:**
 - The Laravel Application sends a “payment request” to the simulated service when a user confirms a booking.
 - The simulation responds with a mock transaction status.
 - The result is stored in the MySQL Database for consistency and reporting, just like a real transaction.

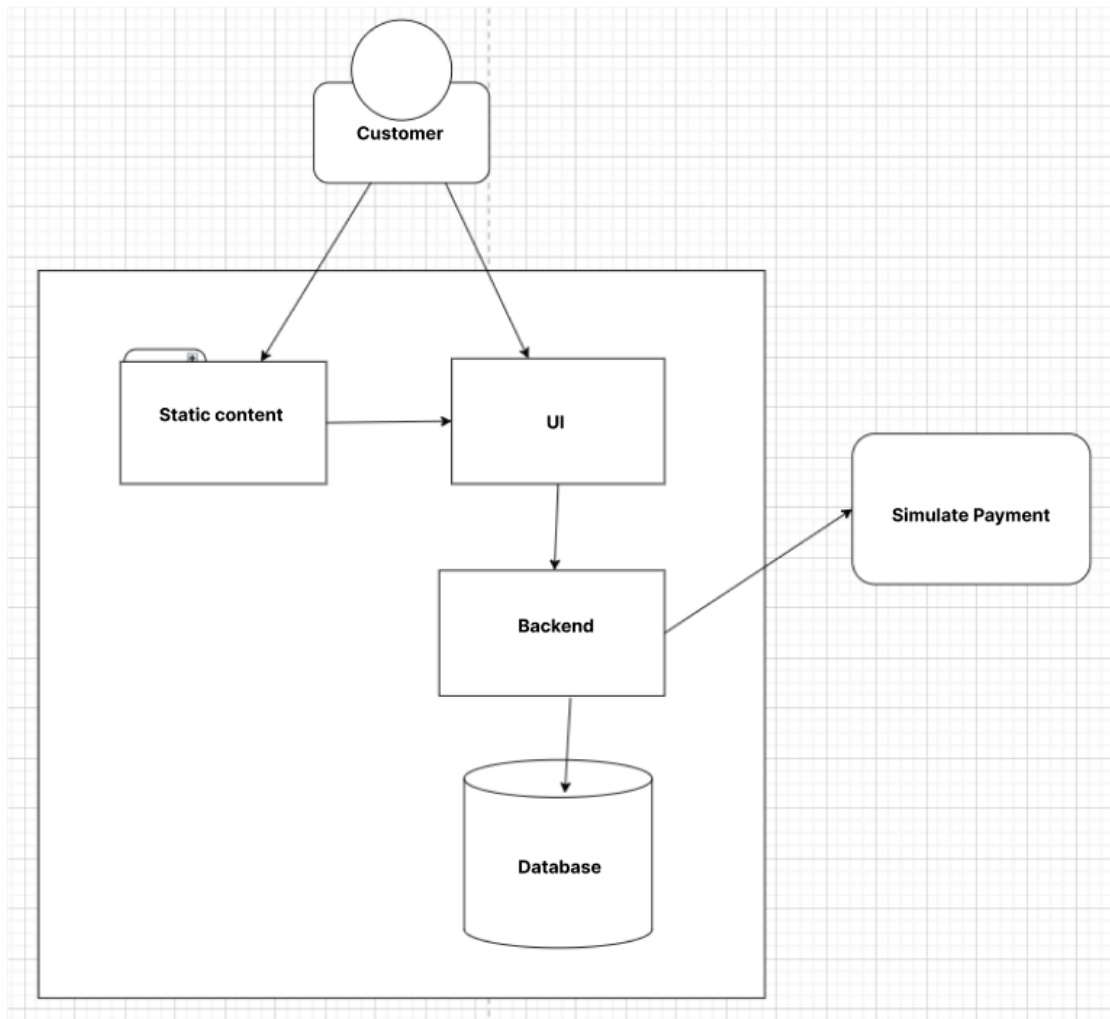


Figure 3: Container Diagram

Component Diagram (C4 Model – Level 3)

◆ Web Browser (UI Container)

Components:

- **HTML Renderer:** Displays dynamic pages generated by Blade templates.
- **Form Handler:** Submits booking forms, login forms, and administrative actions.
- **Session Manager:** Maintains user sessions and cookies for authentication.
- **Request Dispatcher:** Sends HTTP requests (GET/POST) to the backend API.

Interactions: Users interact with the Web Browser, which sends requests to the API layer of the Laravel Application and renders responses (HTML or JSON).

◆ MySQL Database (Data Container)

Components:

- **User Table:** Stores user accounts and authentication data.
- **Movie Table:** Contains movie details (title, genre, duration).
- **Showtime Table:** Stores showtime schedules linked to cinema rooms.
- **Seat Table:** Defines seat layout and availability.
- **Booking Table:** Records booking transactions.
- **BookingSeat Table:** Maps seats to bookings.
- **Report Data:** Aggregated data for statistical reporting.

Interactions: Models in Laravel Application query and update these tables through Eloquent ORM, ensuring transactional integrity.

◆ Payment Simulation (External Container)

Components:

- **Request Handler:** Receives simulated payment requests from the backend.
- **Validation Module:** Mimics card/e-wallet validation.
- **Transaction Processor:** Returns mock transaction results (success/failure).
- **Response Generator:** Sends results back to the Laravel Application.

Interactions: The BookingService calls the Payment Simulation module during booking confirmation. The module responds with simulated outcomes, which are stored in the database for consistency.

◆ **Laravel Application (Backend Container)**

Within the Laravel Application (Monolithic Backend) container, the system is divided into several core components. Each component has a specific responsibility and interacts with others to ensure smooth processing of business operations and communication with external containers such as the Web Browser (UI) and the Database.

Main Components

- **API Layer**
 - Acts as the entry point for all HTTP requests coming from the Web Browser.
 - Routes requests to the appropriate controllers.
 - Returns responses in either JSON (for API endpoints) or HTML (for web pages).
- **Controllers**
 - BookingController: Manages the booking workflow, including seat selection and validation.
 - MovieController: Handles movie listings and details.
 - AdminController: Provides administrative functions such as user management, showtime management, and reporting.
 - Controllers coordinate with services to execute business logic.
- **Models / Eloquent ORM**
 - Represent system entities such as User, Movie, Showtime, Booking, and Seat.
 - Manage database queries and relationships using Laravel's Eloquent ORM.
 - Ensure data consistency and integrity in the MySQL database.

- **Payment Simulation Module**

- Simulates the payment process for testing and demonstration purposes.
- Returns mock transaction results (success/failure) to validate the booking workflow.
- Allows the system to test payment integration without connecting to a real gateway.

- **Views / Blade Templates**

- Render dynamic HTML pages based on data provided by controllers.
- Present information such as movie lists, booking confirmations, and reports to users and administrators.
- Serve as the bridge between backend logic and the Web Browser (UI).

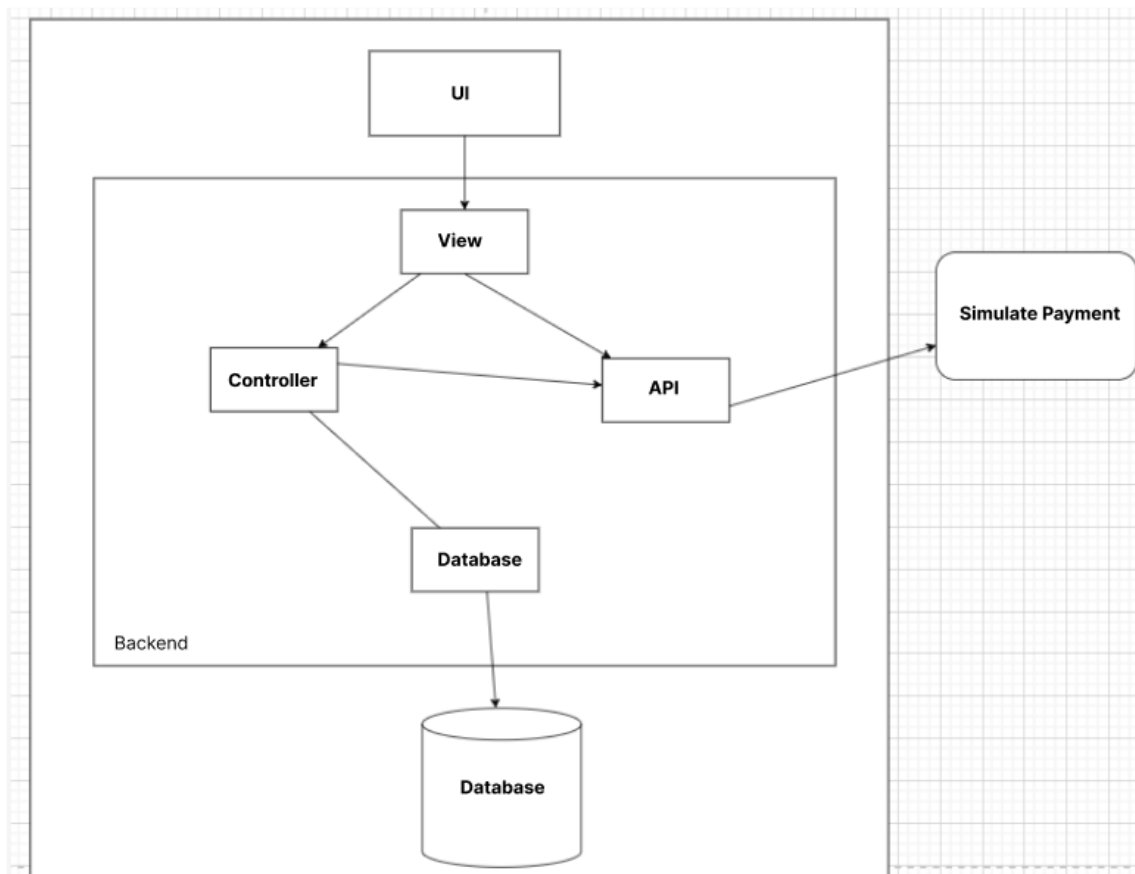


Figure 4: Backend Component Diagram

Code level (C4 Model – Level 4)

◆ Example: Booking Flow

BookingController

- *index()* → Show booking form.
- *store*(Request \$request) → Validate input, call Booking model to create a new booking, update seat status.
- *confirm*(\$id) → Retrieve booking details and pass them to the view.

Booking (Model – Eloquent ORM)

- *fillable* = ['user_id', 'showtime_id', 'seat_id', 'status'] → Defines attributes for mass assignment.
- *user()* → Relationship with User model.
- *showtime()* → Relationship with Showtime model.
- *seat()* → Relationship with Seat model.

Seat (Model – Eloquent ORM)

- *fillable* = ['room_id', 'number', 'status'].
- *isAvailable()* → Check if seat is free.
- *book()* → Update seat status to reserved.

Blade Views

- *booking/index.blade.php* → Displays booking form.
- *booking/confirm.blade.php* → Shows booking confirmation.

◆ Example: Movie Flow

MovieController

- *index()* → List all movies.
- *show*(\$id) → Show movie details.

Movie (Model – Eloquent ORM)

- *fillable* = ['title', 'genre', 'duration'].
- *showtimes()* → Relationship with Showtime model.

Blade Views

- `movies/index.blade.php` → Displays movie list.
- `movies/show.blade.php` → Displays movie details.

◆ Example: Admin Flow

AdminController

- `index()` → Admin dashboard.
- `users()` → Manage users.
- `reports()` → Generate reports from Booking model.

User (Model – Eloquent ORM)

- `fillable = ['name', 'email', 'password']`.
- `bookings()` → Relationship with Booking model.

Blade Views

- `admin/index.blade.php` → Dashboard view.
- `admin/reports.blade.php` → Report view.

◆ Payment Simulation (MVC Integration)

Since you don't use a service layer, the Payment Simulation can be integrated directly in the BookingController:

- `processPayment($bookingId)` → Calls a helper or a simple class `PaymentSimulation` that returns mock results.
- Result is stored in the Booking model and displayed via Blade view.

4.2 Technical Stack and Data Model

- **Server Framework:** Laravel (PHP), a robust MVC-based web framework that provides structured routing, authentication, and database abstraction through Eloquent ORM, enabling rapid development and maintainable code organization.
- **Client:** HTML, CSS, and JavaScript, used to build a responsive web interface for users and administrators to interact with the system through standard HTTP requests.

- **Database:** MySQL (Relational Database), used to persist core system data including users, movies, showtimes, cinema rooms, seats, bookings, and ticket transactions.
- **Data Model:** The system employs a centralized relational data model designed to ensure data consistency and integrity. Key entities include User, Movie, Showtime, CinemaRoom, Seat, Booking, and Ticket. Relationships between entities are enforced through foreign keys and managed using Laravel's Eloquent ORM.

4.3 API Endpoints

1. User / Authentication

Method	Endpoint	Description
POST	/api/register	Register a new user
POST	/api/login	Authenticate user and return token/session
POST	/api/logout	Logout current user

Table 2: User API

2. Movies

Method	Endpoint	Description
GET	/api/movies	List all movies
GET	/api/movies/coming-soon	List coming soon movies
GET	/api/movies/showing	List showing movies
GET	/api/movies/{id}	Get details of a specific movie

Table 3: Movie API

3. Showtimes

Method	Endpoint	Description
GET	/api/showtimes	List all showtimes
GET	/api/showtimes/{id}	Get details of a specific showtime, including available seats

Table 4: Showtime API

4. Booking

Method	Endpoint	Description
POST	/api/bookings	Create a new booking for selected seats
GET	/api/bookings/combo	Create a new booking for selected seats along with combo items (e.g., popcorn, drinks).
GET	/api/booking/{id}	Get details of a booking

Table 5: Booking API

5. Seats

Method	Endpoint	Description
GET	/api/showtimes/{id}/seats	List all seats for a showtime with availability status

Table 6: Seat API

6. Admin

Method	Endpoint	Description
GET	/api/admin/bookings	List all bookings
GET	/api/admin/users	List all registered users
GET	/api/admin/movies	List all movies
POST	/api/admin/movies	Create a new movie
PUT	/api/admin/movies/{id}	Update movie details
GET	/api/admin/showtimes	List all showtimes
POST	/api/admin/showtimes	Create a new showtime

Table 7: Admin API

4.4 Overall Architecture – Layered Monolith

The system follows a layered monolithic architecture, where all functionality is packaged into a single deployable unit but logically separated into distinct layers. This approach ensures clear separation of concerns, maintainability, and consistency.

◆ **Presentation Layer**

- **Role:** Provides the user interface and handles interactions between users and the system.
- **Components:**
 - **Web Browser (UI):** Displays dynamic HTML pages rendered by Blade templates.
 - **Blade Views:** Generate and render HTML pages for booking, account management, and administrative tasks.
 - **Controllers:** Receive HTTP requests from the browser, validate input, and forward requests to the business layer.
- **Responsibilities:**
 - Manage user sessions and authentication.
 - Handle form submissions (booking, login, administration).
 - Present results and feedback to users.

◆ **Business Layer**

- **Role:** Encapsulates the core business logic of the system.
- **Components:**
 - **Controllers (logic coordination):** Orchestrate workflows such as booking or reporting.
 - **Business Logic (embedded in Controllers/Models):**
 - Seat availability checks.
 - Booking creation and ticket issuance.
 - Report generation for administrators.

- **Payment Simulation Module:** Mimics payment transactions and returns mock results.
- **Responsibilities:**
 - Enforce business rules and workflows.
 - Coordinate between presentation and database layers.
 - Ensure transactional consistency during booking and payment processes.
- ◆ **Database Layer**
- **Role:** Provides persistent storage and ensures data integrity.
- **Components:**
 - **MySQL Database:** Stores all entities including users, movies, showtimes, cinema rooms, seats, bookings, and tickets.
 - **Eloquent ORM Models:** Represent database tables as objects and manage relationships.
- **Responsibilities:**
 - Handle CRUD operations for all entities.
 - Maintain referential integrity between tables (e.g., bookings linked to seats and showtimes).
 - Support transaction management to prevent double-booking or inconsistent states.

4.5 Detailed Architecture

The detailed architecture of the Cinema Management System illustrates how the system's core components interact to fulfill business operations, including movie browsing, seat selection, booking, and administrative reporting.

Core Components

1. Web Browser / Client

- Users and administrators interact with the system through a web browser.
- Requests are sent via **HTTP** (GET or POST) to the Laravel backend.

- Pages are rendered using **Blade templates**, showing dynamic content such as movie lists, seat availability, and booking confirmations.

2. Laravel Application (Monolithic Backend)

- **Controllers** handle incoming requests and coordinate application logic.
 - BookingController: manages seat selection, booking creation,
 - MovieController: manages movie listing and details.
 - ShowtimeController: manages showtimes and seat availability.
 - AdminController: generates reports and manages users, rooms, and showtimes.
- **Models / Eloquent ORM** represent entities in the database:
 - User, Movie, Showtime, Seat, Room , Booking, BookingSeet

BookingCombo

3. Database (MySQL)

- Stores all persistent data: users, movies, showtimes, cinema rooms, seats, bookings, combo items.
- Supports **transactional integrity**, ensuring seat booking is atomic and preventing double booking.
- Relationships between entities are managed via foreign keys and Eloquent ORM.

◆ Flow 1: Movie Booking

1. User Interaction

- A user navigates to the booking page via the **Web Browser (UI)**.
- The interface displays available movies, showtimes, and seat layouts using **Blade templates**.
- The user selects seats and optionally adds combo items (e.g., popcorn, drinks).

2. Form Submission

- The browser sends a **POST request** to BookingController@store.

- Request parameters include user ID, showtime ID, selected seats, and combo items.

3. Server Processing

- **BookingController** validates input and checks seat availability.
- A **database transaction** begins:
 - A new **Booking** record is created.
 - **BookingSeat** entries are inserted for each selected seat.
 - **BookingCombo** entries are recorded if applicable.
 - The total price is calculated.
- If all steps succeed, the transaction commits; otherwise, it rolls back to prevent inconsistencies.

4. Response

- The system returns a **confirmation page** (Blade template) or a JSON response for API endpoints.
- The UI updates to reflect booked seats and displays a success message.

◆ Flow 2: Payment Processing (Simulation)

1. Payment Request

- After booking creation, the user proceeds to payment.
- The **BookingController** calls the **Payment Simulation Module** instead of a real gateway.

2. Payment Simulation

- The module mimics a real payment gateway by validating the request and returning a mock result (success/failure).
- No external API is involved; the simulation ensures the booking workflow can be tested end-to-end.

3. Database Update

- If the simulated payment succeeds:
 - The **Booking** record is updated with payment status = “confirmed.”
 - Transaction details are stored for reporting.

- If payment fails:
 - The booking remains pending or is canceled.

4. Response

- The system returns a **payment confirmation page** (Blade template) or JSON response.
- The UI reflects the payment outcome and provides feedback to the user.

4.6 Implementation: Server-Side Logic (Laravel)

The server-side logic is responsible for handling core business operations such as movie browsing, seat selection, booking validation, and booking confirmation. These operations are implemented using Laravel controllers and models following the MVC pattern.

Request Handling

User requests are received via HTTP routes and processed by controllers. Each controller coordinates the required business logic, interacts with the data layer through Eloquent models, and returns appropriate responses to the client.

// Example: BookingController handling seat booking

```
public function store(Request $request)
{
    $request->validate([
        'showtime_id' => 'required|exists:showtimes,id',
        'seat_ids' => 'required|array|min:1',
        'seat_ids.*' => 'exists:seats,id',
    ]);

    DB::beginTransaction();

    try {
```

```
        $showtime = Showtime::with('room.seats')->findOrFail($request->showtime_id);
```

```
$bookedSeats = BookingSeat::whereHas('booking', function($q) use ($request) {
```

```
    $q->where('showtime_id', $request->showtime_id)
        ->where('status', 'confirmed');
    }->whereIn('seat_id', $request->seat_ids)->pluck('seat_id')->toArray();
```

```
if (!empty($bookedSeats)) {
    DB::rollBack();
    return response()->json([
        'success' => false,
        'message' => 'Some seats are already booked',
        'booked_seats' => $bookedSeats
    ], 409);
}
```

```
$booking = Booking::create([
    'user_id' => auth()->id(),
    'showtime_id' => $request->showtime_id,
    'total_amount' => 0,
    'status' => 'confirmed',
]);
```

```
$total = 0;
```

```
foreach ($request->seat_ids as $seatId) {
    $seat = $showtime->room->seats->where('id', $seatId)->first();
```

```

$price = 36000; // default regular
if ($seat) {
    if (strpos($seat->seat_code, 'L') === 0) {
        $price = 90000; // Couple seats
    } elseif (in_array($seat->seat_code[0], ['D', 'E', 'F', 'G', 'H'])) {
        $price = 49000; // VIP seats
    }
}

BookingSeat::create([
    'booking_id' => $booking->id,
    'seat_id' => $seatId,
    'showtime_id' => $request->showtime_id,
    'price' => $price,
    'status' => 'confirmed',
]);

$total += $price;
}

$booking->update(['total_amount' => $total]);

DB::commit();

return response()->json([
    'success' => true,
    'booking_id' => $booking->id,
    'total_amount' => $total,

```

```

        'message' => 'Booking created successfully'
    ];

    } catch (\Throwable $e) {
        DB::rollBack();
        return response()->json([
            'success' => false,
            'message' => 'An error occurred: ' . $e->getMessage()
        ], 500);
    }
}

```

Flow

When a client sends a request (e.g., booking a seat), the corresponding controller method is executed. The system validates input data, applies business rules such as seat availability checks, updates the database accordingly, and returns a response to the client. This synchronous request–response flow ensures data consistency and reliable processing of critical booking operations.

4.7 Implementation: Client-Side Logic (Laravel Blade / Server-Side Rendering)

The client-side logic is implemented using **Blade templates**. Users interact with the system via HTML forms and hyperlinks to perform operations such as seat selection and booking confirmation. No JavaScript or WebSocket is required, as the system relies on standard HTTP request-response for updating the interface.

Form and UI Update Example:

```

<!-- resources/views/bookings/create.blade.php -->
<form action="{{ route('bookings.store') }}" method="POST">
    @csrf
    <input type="hidden" name="showtime_id" value="{{ $showtime->id }}">
    <label for="seat">Select Seat(s):</label>
    <select name="seat_ids[]" id="seat" multiple>

```

```

@foreach($showtime->room->seats as $seat)
    <option value="{{ $seat->id }}"
        @if(in_array($seat->seat_code, $soldSeatCodes)) disabled @endif>
        {{ $seat->seat_code }}
    </option>
@endforeach
</select>
<button type="submit">Book Seat</button>
</form>
@if(session('success'))
    <div class="alert alert-success">{{ session('success') }}</div>
@endif

```

Flow

1. A user selects seat(s) and submits the form.
2. The request is sent to the Laravel backend (BookingController@store).
3. The server validates the input, checks seat availability, creates the booking, and returns a response.
4. The page reloads or redirects with a success message, updating the UI to reflect booked seats.

5. Testing & Verification

The primary verification for this project is to ensure that the core functionalities—seat selection, booking, and data persistence—work correctly and reliably.

Verification Proof: Booking Functionality

Test Scenario: A user selects seats and submits a booking request.

Step	Action	Expected Result	Actual Result	Status
1	Open Browser as User A	Booking page for a specific showtime loads successfully	Booking page loads with available seats	Pass
2	Select one or more available seats and submit booking form	Seats are validated; booking is created in the database; user sees a confirmation message	Booking created successfully; confirmation displayed	Pass
3	Open Browser as User B and check the same showtime	Previously booked seats are no longer available	Booked seats disabled / marked as sold	Pass
4	Attempt to book already booked seats	System prevents booking and shows an error message	Error message displayed: "Some seats are already booked"	Pass

Table 8: Testing

Verification Evidence

- Screenshots of the booking page showing selected seats marked as booked.
- Database query confirming that Booking and BookingSeat records have been created.
- User interface correctly prevents double booking and displays confirmation messages.

6. Conclusion & Reflection

This project successfully implemented a Cinema Management System using a Monolithic Laravel architecture. Core functionalities including movie browsing, seat selection, booking, and administrative reporting were fully implemented. The system ensures data integrity and prevents double booking through server-side validation and transactional handling. Verification confirmed that the booking workflow operates correctly, with the database updated accurately and the user interface reflecting seat availability.

The project demonstrates that a **Monolithic MVC approach** is effective for managing cinema operations, providing a simple, cohesive, and reliable solution for both customers and administrators.

6.1 Lessons Learned

- **Transactional Integrity:** Using Laravel's database transactions ensures that bookings are processed safely, avoiding inconsistencies when multiple users book seats simultaneously.
- **Server-Side Validation:** Implementing validation in controllers prevents invalid or duplicate bookings, maintaining data consistency.
- **MVC Architecture Simplicity:** The Monolithic Laravel MVC structure allows rapid development and straightforward maintenance for small to medium-scale systems.
- **UI Feedback:** Blade templates provide clear feedback to users about booking status, improving user experience without requiring client-side scripting.

6.2 Future Improvements

1. **Enhanced Security:** Implement stronger user authentication and authorization mechanisms, such as Laravel Sanctum or Passport, to protect user accounts and administrative operations.

2. **Performance & Scalability:** Introduce caching (e.g., Redis) for frequently accessed data like showtimes and seat availability to improve response times during peak booking periods.
3. **Extended Features:** Add features like booking history for users, email notifications for successful bookings, and dynamic reporting for administrators to provide a better user experience and more actionable insights.
4. **Deployment & Reliability:** Deploy the system on a cloud platform (e.g., AWS or DigitalOcean) with automated backups and monitoring to ensure high availability and data safety.

References

1. Rozanski, N., & Woods, E. (2011). *Software Architecture: Visual Lecture Notes*. Springer.
2. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
3. Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley.